

United Airlines Flight Delay Analysis

A Beginner-Friendly Walkthrough of the Project — 6 Lessons

Written for anyone learning data analysis by following this real project
step by step — no prior experience assumed.

What This Document Covers

This document explains, in plain language, everything that was built for the **United Airlines Flight Delay & Operational Performance Analysis** project — a data analyst portfolio project. Each lesson builds on the one before it, the same way the project itself was built, step by step.

1. Project Structure — why we organized the files the way we did
2. The Data-Cleaning Script — turning messy raw data into something usable
3. The Exploratory Analysis Script — asking questions of the data with code
4. Interpreting Findings — turning numbers into a story a business can act on
5. The SQL Layer — doing the same analysis a second way, using databases
6. Git & GitHub — saving and sharing the work properly

No prior coding or data experience is assumed. Every technical term is explained the first time it appears.

Lesson 1 — Project Structure

Before writing a single line of analysis code, the first decision on any data project is organization: where do files live, and why. A messy folder full of randomly named files looks unprofessional to anyone who opens it — including a recruiter looking at your GitHub page. So the very first thing we did was set up a clear, standard folder layout.

```
United-Airlines-Data-analysis/  
├─ data/  
│   ├── raw/          <- the original, untouched downloaded data  
│   └─ processed/     <- the cleaned data our scripts produce  
├─ notebooks/  
│   ├── 01_data_cleaning.py  
│   ├── 02_exploratory_analysis.py  
│   └─ figures/       <- charts our scripts generate  
├─ sql/  
│   ├── load_to_sqlite.py  
│   └─ queries.sql  
├─ dashboard/        <- reserved for a future visual dashboard  
├─ ai_assistant/      <- reserved for future AI features  
├─ README.md  
├─ requirements.txt  
└─ .gitignore
```

Why "raw" and "processed" are kept separate

This is a rule almost every professional data analyst follows: **never edit the original data directly**. The raw file always stays untouched. Every cleaning step happens in code, and the result is saved somewhere new (into `processed/`). The benefit: if a mistake is made while cleaning the data, nothing is lost — you can simply re-run the cleaning code from the original file again. If you'd overwritten the original, a mistake would be permanent and unrecoverable.

Why the scripts are numbered

Files named `01_data_cleaning.py` and `02_exploratory_analysis.py` tell anyone opening the project — including a future version of yourself — the exact order things must run in. Step 2 depends on the output of Step 1, so the numbers make that dependency obvious without needing to read any code.

What a ".gitignore" file is for

A `.gitignore` file is a list of files and folders you tell Git (the tool that tracks your project's history — explained fully in Lesson 6) to *never* track. We used it to exclude:

- The raw data file — 600 megabytes, far too large to store online comfortably
- The generated database file — 51 megabytes, easily regenerated by re-running a script

The rule of thumb: **code belongs in version control; large data files do not.** Anyone who downloads this project can regenerate both files by simply running the scripts against the original download.

Why the README file matters most

When someone opens this project on GitHub, the `README.md` file is the very first thing they see — often the *only* thing they read before deciding whether to look further. Everything else in the project exists to support and prove what the README claims.

Key takeaway: Good organization is not a cosmetic detail — it is itself a skill being evaluated. Separating raw data from processed data, numbering steps in order, and keeping large files out of version control are all small decisions that, together, signal that you understand how real projects are run.

Lesson 2 — The Data-Cleaning Script

Real-world data is never ready to use straight out of the box. Our starting file contained 3 million flight records from every U.S. airline combined. Before we could ask any question about United specifically, the data had to be filtered, corrected, and organized. This is called **data cleaning**, and skipping it leads to wrong or misleading results later — a principle often summarized as "garbage in, garbage out."

Working with file paths that work on any computer

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
RAW_PATH = PROJECT_ROOT / "data" / "raw" / "flights_sample_3m.csv"
```

Instead of typing a fixed path like `C:\Users\...`, the script calculates its own location and works upward from there. This means the exact same script would work correctly if copied to a different computer, or if someone else downloaded the project from GitHub — nothing is hardcoded to one specific machine.

Loading the data

```
df = pd.read_csv(RAW_PATH)
```

Pandas is a Python toolkit built specifically for working with tabular data — rows and columns, like a spreadsheet, but controlled entirely through code instead of mouse clicks. `pd.read_csv(...)` reads a CSV (comma-separated values) file into a structure called a **DataFrame**, which is pandas' core building block.

Filtering down to United Airlines only

```
ua = df[df["AIRLINE_CODE"] == "UA"].copy()
```

This line does something used constantly in data analysis: it creates a "True/False" test for every single row (is this row's airline code equal to "UA"?), then keeps only the rows where that test came back True. Out of 3 million total flights, roughly 254,500 belonged to United — everything else was discarded from this working copy.

Creating new, more useful columns

```
ua["FL_DATE"] = pd.to_datetime(ua["FL_DATE"])
ua["YEAR"] = ua["FL_DATE"].dt.year
ua["MONTH"] = ua["FL_DATE"].dt.month
```

The flight date originally exists only as plain text (for example, "2019-01-09"). Converting it into a proper date value lets the computer understand it as an actual calendar date rather than a string of characters — which then makes it trivial to pull out just the year or just the month for every row. This step of creating new, more useful columns out of existing data is called **feature engineering**, and it comes up constantly in analysis work.

Deciding what a "missing" value actually means

```
ua[delay_cause_cols] = ua[delay_cause_cols].fillna(0)
```

Some columns in the data (like "minutes delayed due to weather") were often blank. It would be a mistake to assume blank means "unknown" and throw those rows away — in this dataset, a blank simply means that cause contributed zero minutes to that flight's delay. Filling those blanks with the number 0 is the correct interpretation here. This is an important general lesson: **always figure out what a missing value actually represents before deciding how to handle it** — guessing wrong can silently delete large amounts of perfectly good data.

Creating a simple delay flag

```
ua["IS_DELAYED_15"] = ua["ARR_DELAY"] >= 15
```

This creates a new column that is simply True or False: was this flight delayed by 15 minutes or more on arrival? Fifteen minutes is not an arbitrary number — it is the official threshold the U.S. Federal Aviation Administration uses to define an "on-time" flight, so using it here keeps our results comparable to official statistics.

Saving the cleaned result

```
ua.to_csv(PROCESSED_PATH, index=False)
```

The cleaned data — now containing only United flights, with proper dates and filled-in delay columns — is saved as a brand-new file, leaving the original raw file completely untouched.

Key takeaway: Cleaning data is not a mechanical chore — it is a series of judgment calls (what to filter, how to interpret missing values, what new columns are worth creating) that a good analyst should always be able to explain and justify.

Lesson 3 — The Exploratory Analysis Script

With clean data ready, this step is where we actually answer questions: which airports have the worst delays, which months are worst, what causes the delays, and whether performance is improving or getting worse over time.

Excluding flights that never actually arrived

```
flown = df[~df["CANCELLED"] & ~df["DIVERTED"]].copy()
```

The symbol `~` means "not." This line keeps only flights that were *not* cancelled and *not* diverted, because a cancelled flight never had a real arrival time — including it would distort any calculation of "average delay."

The most important pattern in data analysis: group, then measure

```
airport_delay = (  
    flown.groupby("ORIGIN")["ARR_DELAY"]  
    .agg(avg_delay="mean", flights="count")  
    .query("flights >= 200")  
    .sort_values("avg_delay", ascending=False)  
)
```

Read this step by step:

1. **Group** every flight by which airport it departed from.
2. **Measure** two things inside each group: the average arrival delay, and how many flights are in that group.
3. **Filter out small groups** — an airport with only a handful of flights could show a misleading average, so anything under 200 flights is dropped.
4. **Sort** from worst average delay to best.

This exact sequence — group, measure, filter, sort — is reused for almost every other question in the project, just changing what column is grouped by.

Looking at a trend across months

```
monthly_delay = flown.groupby("MONTH")["ARR_DELAY"].mean().sort_index()
```

Same pattern as above, but grouped by month instead of airport, and sorted back into calendar order (January through December) rather than by delay size — because for a trend, chronological order is what makes it readable.

Comparing several causes side by side

```
cause_totals = flown[delay_cause_cols].sum().sort_values(ascending=False)
```

The dataset records five separate columns for delay minutes — one each for carrier, weather, air-traffic-control, security, and late-aircraft causes. This line adds up each of those columns separately across all 254,000+ flights, producing one grand total per cause, which can then be ranked.

Turning numbers into charts

```
fig, ax = plt.subplots()
sns.barplot(x=..., y=..., ax=ax)
fig.savefig("chart.png", dpi=150)
```

Matplotlib is the standard Python library for drawing charts; **Seaborn** builds on top of it to make good-looking charts easier to produce. Each chart in the project follows the same four-step recipe: group and measure the data, draw it, label it clearly, and save it as an image file that can be dropped straight into a report.

Key takeaway: Once you're comfortable with the "group, measure, filter, sort" pattern, most everyday analysis questions become a matter of assembling that same pattern with different columns — this really is the core of what a data analyst does day to day.

Lesson 4 — Interpreting Findings and Writing Recommendations

This lesson has no new code — it covers a skill that matters just as much as writing the code: turning raw numbers into something a business could actually act on. This is the difference between "I can run pandas commands" and "I can think like an analyst."

The three-part structure behind every finding

Every finding in the project's README follows the same shape:

Part	Example
1. The claim	"Late-aircraft delays are UA's #1 problem, not weather."
2. The evidence	"40.6% of delay-minutes come from late aircraft, versus only 5.2% from weather."
3. The "so what"	"This means most delay-minutes are operationally controllable — better scheduling and turnaround management would have real, measurable impact."

Most people stop at part 2 — reporting a number. The third part, translating that number into a business consequence, is the part that actually demonstrates analytical thinking, and it is the part a hiring manager notices.

Every recommendation must trace back to a finding

Notice that each recommendation in the project directly matches a finding above it — for example, the finding that late-aircraft and carrier delays make up 71% of delay-minutes directly produces the recommendation to "prioritize aircraft turnaround over weather mitigation." If a recommendation cannot be traced back to a specific number, it is just an opinion — and in an interview, that is usually the first thing someone will challenge you on.

Distinguishing what a company can and cannot control

One detail that makes the analysis feel like real industry knowledge rather than a generic exercise: we separated delay causes into "controllable" (late aircraft, carrier issues — things United's own operations affect) versus "uncontrollable" (weather, air

traffic congestion). Framing findings this way signals an understanding of how the airline industry actually thinks about its own performance.

Key takeaway: Analysis is not complete once a chart exists. It is complete once you can answer, in one sentence, "so what should the business do differently because of this number?"

Lesson 5 — The SQL Layer

Having already answered these questions once using pandas, we then answered the exact same questions a second way, using SQL. This was intentional, not repetitive busywork.

Why do the same analysis twice?

Most companies store their real data inside databases, not spreadsheets or CSV files, and a large share of everyday analyst work involves writing SQL directly against those databases. Showing both pandas and SQL skills on the same project proves you're not limited to a single tool. It also let us double-check our own work — when the pandas answer and the SQL answer matched exactly, that gave real confidence the analysis was correct.

What SQLite is

SQLite is a genuine, fully functional SQL database that lives inside a single file on disk, rather than requiring a separate server to be installed and running. That makes it ideal for a project like this one — no setup required, just a `.db` file.

Getting data into the database

```
df.to_sql("flights", conn, if_exists="replace", index=False)
```

This line takes the cleaned data (already prepared in Lesson 2) and writes it into a database table named "flights" — the bridge connecting the pandas world to the SQL world.

Reading a SQL query, side by side with pandas

```
SELECT
  ORIGIN,
  COUNT(*)                AS num_flights,
  ROUND(AVG(ARR_DELAY), 1) AS avg_arrival_delay_min
FROM flights
WHERE CANCELLED = 0 AND DIVERTED = 0
GROUP BY ORIGIN
HAVING COUNT(*) >= 200
```

```
ORDER BY avg_arrival_delay_min DESC
LIMIT 10;
```

SQL piece	What it does	Equivalent pandas idea
WHERE	Filter rows before grouping	Filtering the DataFrame first
GROUP BY	Bucket rows by airport	<code>.groupby()</code>
AVG() , COUNT()	Measure each bucket	<code>.agg()</code>
HAVING	Filter buckets after grouping	<code>.query()</code> after grouping
ORDER BY	Sort the results	<code>.sort_values()</code>
LIMIT	Keep only the top rows	<code>.head()</code>

A detail worth remembering: `WHERE` filters individual rows before any grouping happens, while `HAVING` filters entire groups after they've been calculated. You cannot write `WHERE COUNT(*) >= 200` because, at the point `WHERE` runs, no grouping or counting has happened yet. This distinction is a very common question in interviews.

Key takeaway: SQL and pandas are two different syntaxes expressing the same underlying logic — filter, group, measure, filter again, sort, limit. Understanding that logic once means you can apply it in either language.

Lesson 6 — Git and GitHub

The final piece of the project is version control — the system that saved the work properly and made it visible to others.

Git versus GitHub — two different things

- **Git** is a program running on your own computer that tracks every change made to your project's files over time, saved as snapshots called "commits." It works completely offline.
- **GitHub** is a website that stores a copy of that history online, so it's backed up and so other people — including recruiters — can view it.

Git is the tool; GitHub is simply one place to keep what Git tracks.

The everyday workflow, step by step

Command	What it means
<code>git init</code>	Turn an ordinary folder into a Git-tracked project (done once)
<code>git add -A</code>	"Stage" changed files — like putting items in a shopping cart before checkout
<code>git commit -m "message"</code>	Save a permanent snapshot of everything staged, with a note explaining the change
<code>git branch -M main</code>	Name the main line of development "main" (today's standard convention)
<code>git remote add origin <url></code>	Tell Git where the GitHub copy of this project lives
<code>git push -u origin main</code>	Upload the saved history to GitHub

Why the .gitignore file mattered here specifically

Because the raw data file and the generated database were listed in `.gitignore`, they were automatically skipped during `git add` — keeping the GitHub repository small, fast to download, and free of files that don't belong in version control.

Why good commit messages matter

A commit message is the only explanation future readers get for why a change was made, without having to re-read every line of code that changed. Writing a clear, specific message (what changed, and briefly why) is a habit worth building early.

Key takeaway — the core loop to remember:

Edit files → `git add` → `git commit` → `git push` .

This same four-step rhythm is repeated for essentially every future change to any Git-tracked project.

End of the 6-lesson walkthrough — covering project structure, data cleaning, exploratory analysis, interpreting findings, SQL, and Git/GitHub.

Prepared as a companion document to the United Airlines Flight Delay Analysis project.